# НАСЛЕДОВАНИЕ

В процедурном программировании код организуется вокруг функций и алгоритмов, но, когда мы начинаем моделировать реальный мир, появляется необходимость работать с сущностями предметной области. И здесь мы сталкиваемся с интересной закономерностью: многие сущности имеют общую природу, но отличаются конкретными деталями. Например, все люди имеют имя и возраст, но студент отличается от преподавателя, а менеджер — от рядового сотрудника. Все транспортные средства могут передвигаться, но автомобиль отличается от самолёта, а велосипед — от корабля.

Если мы попытаемся описать каждую такую сущность полностью независимым классом, наш код быстро начнёт разрастаться, в нём появятся дублирующиеся участки, а любое изменение в общей логике потребует правок во множестве мест. Именно эту проблему и решает наследование.

Наследование — это механизм объектно-ориентированного программирования, который позволяет одному классу наследовать поля, свойства и методы другого класса.

Класс, от которого происходит наследование, называется:

**базовый класс** (base class, родительский).

Класс, который наследует:

**производный класс** (derived class, дочерний).

Производный класс:

– автоматически получает доступ ко всем доступным членам базового класса;

– может добавлять собственные поля и методы;

– может изменять поведение унаследованных элементов.

Формально наследование можно определить как механизм объектно-ориентированного программирования, при котором один класс расширяет или уточняет другой класс, автоматически получая его структуру и поведение. Но

важно понимать, что наследование — это не просто копирование кода из одного класса в другой. Это установление иерархической связи между классами, которая отражает реальные отношения между сущностями предметной области.

Ключевым принципом, определяющим корректность использования наследования, является семантика отношения «is-a» (является). Наследование уместно только тогда, когда между классами существует именно такое отношение.

Например, утверждение «Студент является Человеком» звучит логично, значит, класс Student может наследовать от класса Person. Аналогично «Автомобиль является ТранспортнымСредством» или «Администратор является Пользователем». Если же подобное утверждение звучит странно или натянуто — скорее всего, наследование выбрано неправильно. Это архитектурное, а не синтаксическое правило, но именно его нарушение чаще всего приводит к проблемам в дизайне системы.

Важно понимать, что производный класс — это частный случай базового класса.

В языке C# наследование классов объявляется с помощью **оператора двоеточия**.

*Общий синтаксис:*

```
class BaseClass
{
   // базовый класс
}

class DerivedClass : BaseClass
{
   // производный класс
}
```

*Конкретный пример:*

```
class Person
{
   public string Name;
   public int Age;
}

class Student : Person
{
   public string Group;
}

class Teacher : Person
{
   public string Subject;
}
```

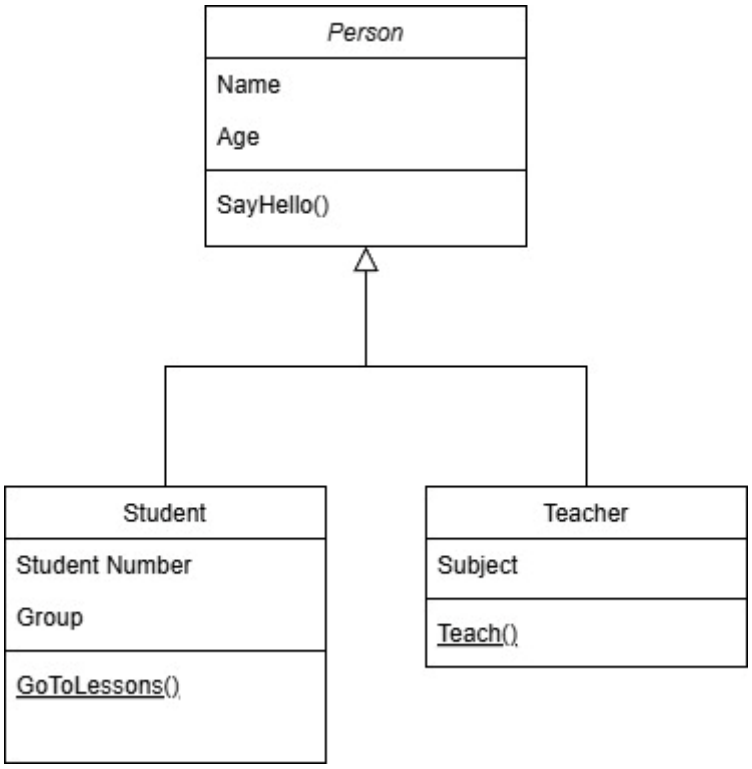
Здесь мы говорим компилятору, что класс Student расширяет класс Person. Это означает, что каждый объект типа Student автоматически содержит все поля и методы, определённые в классе Person, плюс собственное поле Group. Аналогично, класс Teacher наследует от Person и добавляет поле Subject.

Без использования наследования нам пришлось бы дублировать поля Name и Age в обоих классах, что привело бы к избыточности кода и сложностям в его поддержке. Без наследования код выглядел бы так:

```
class Student
{
   public string Name;
   public int Age;
}

class Teacher
{
   public string Name;
   public int Age;
}
```

Например, если бы мы захотели изменить тип поля Name с string на специальный класс FullName, пришлось бы вносить изменения в двух местах. С наследованием же мы меняем только базовый класс Person.

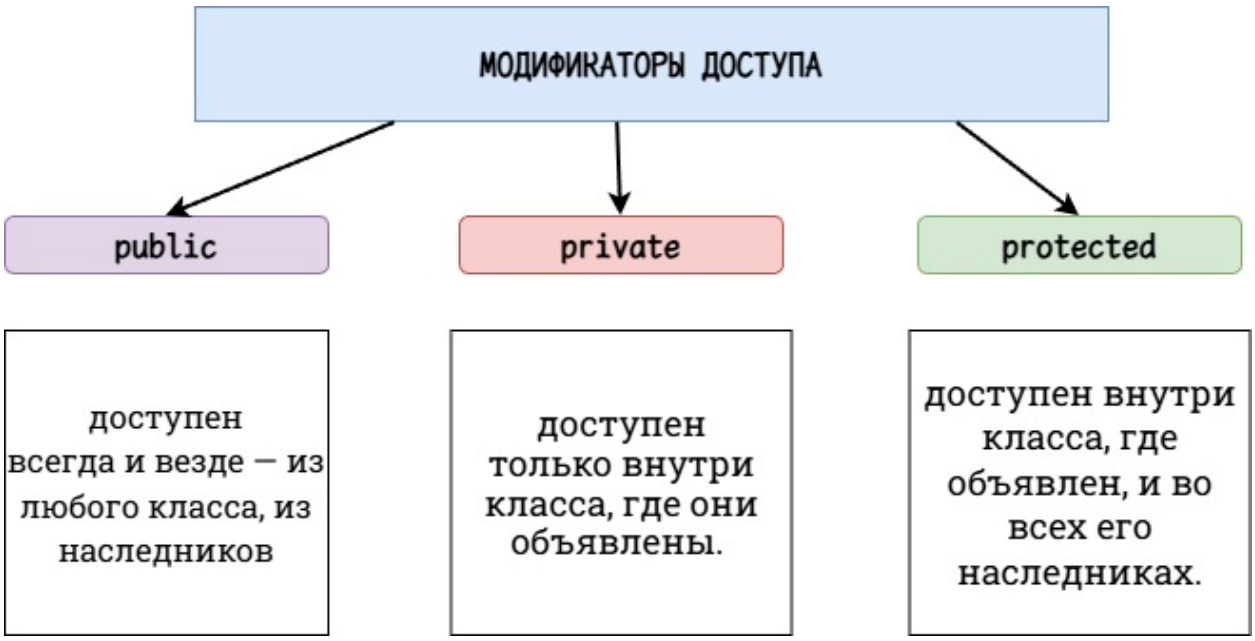
Важно понимать физическую модель наследования. Объект производного класса не просто «знает о» базовом классе — он физически содержит базовый объект как свою составную часть. Условно структуру объектов Student и Teacher можно представить так:




Оба класса Student и Teacher наследуются от класса Person и могут использовать поля и методы как свои, так и родительского класса.

Базовый класс всегда инициализируется первым, его состояние всегда существует внутри объекта производного класса, и «отключить» эту базовую часть невозможно. Такая организация обеспечивает целостность объектов и предсказуемость их поведения.

Производный класс получает доступ далеко не ко всем членам базового класса, его ограничивают модификаторы доступа.




**Public**-члены класса доступны всегда и везде: из любого класса, из наследников, из внешнего кода. Они представляют собой открытый интерфейс класса.

**Private**-члены класса доступны только внутри класса, где они объявлены. Даже непосредственный наследник не видит private-члены своего родителя. Это сознательное ограничение, поддерживающее принцип инкапсуляции — внутренняя реализация базового класса скрыта от потомков.

**Protected** члены — это ключевой модификатор для наследования. Protected означает, что член доступен внутри класса, где объявлен, и во всех его наследниках, но не доступен извне. Это оптимальный выбор для данных и методов, которые должны быть доступны потомкам, но не должны быть частью публичного интерфейса.

```
class Person
{
   private int secretCode;      // Не доступен в наследниках
   protected int age;           // Доступен в наследниках
   public string Name;          // Доступен всем
}

class Student : Person
{
   public void PrintAge()
   {
      Console.WriteLine(secretCode); // Ошибка: нет доступа
      Console.WriteLine(age);        // OK: protected доступ
      Console.WriteLine(Name);       // OK: public доступ
   }
}
```

**Конструкторы**

Конструкторы не наследуются в прямом смысле, но при этом каждый объект производного класса обязан корректно инициализировать свою базовую часть. Поэтому конструктор базового класса всегда вызывается при создании объекта производного класса.

Если базовый класс имеет конструктор без параметров, он вызывается автоматически перед выполнением конструктора производного класса:

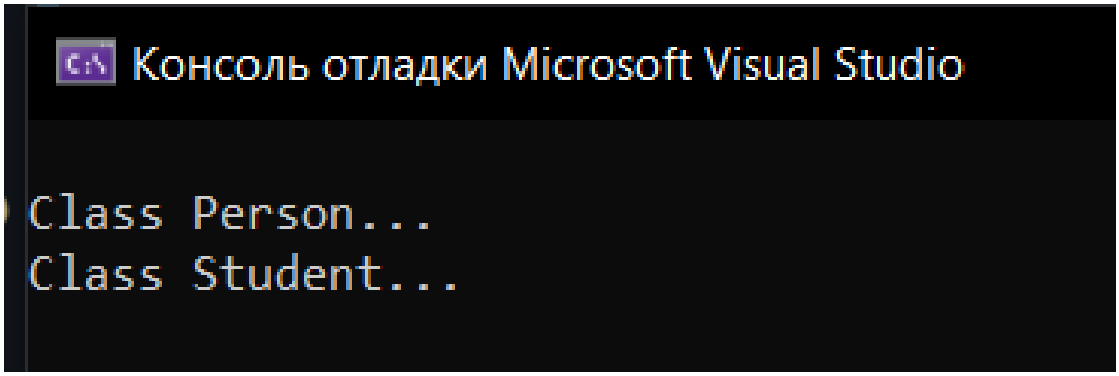
```
Student student = new Student();

class Person
{
   public Person() => Console.WriteLine("Class Person...");
}

class Student : Person
{
   public Student() => Console.WriteLine("Class Student...");
}
```

Здесь при создании Student

1. Вызывается Person()

2. Вызывается Student()

Выведутся оба конструктора




Если же базовый класс не имеет конструктора без параметров, необходимо явно указать, какой конструктор и с какими аргументами нужно вызвать, используя ключевое слово base. По сути, при создании конструктора по умолчанию, компилятор сам добавляет base, поэтому конструктор

```
public Student() => Console.WriteLine("Class Student...");
```

***эквивалентен***

```
public Student() : base()
{
    Console.WriteLine("Создан Student");
}
```

Рассмотрим пример, в котором в родительском классе отсутствует конструктор по умолчанию.

```
class Person
{
    public string Name;

    public Person(string name)
    {
        Name = name;
    }
}
class Student : Person
{
    public Student() {}
}
```

Здесь появится ошибка, так как компилятор пытается вызвать base, однако такого же пустого конструктора в классе Person нет.

Как же быть в такой ситуации: нам необходимо просто явно указать, на что нужно ссылаться.

```
class Person
{
   public string Name;

   public Person(string name)
   {
      Name = name;
   }
}

class Student : Person
{
   public Student(string name) : base(name)  // Явный вызов базового конструктора
   {
      // Дополнительная инициализация для Student
   }
}
```

В конструкторе класса Student мы добавляем параметр, а в скобках слова base мы передаем наш параметр родительскому конструктору, то есть как бы говорим компилятору: «Перед созданием экземпляра класса Student вызови конструктор класса Person, передав ему имя экземпляра».

Рассмотрим более расширенный пример:

```
class Person
{
   public string Name;

   public Person(string name)
   {
      Name = name;
   }
}
```

```
class Student : Person
{
   public int Course;

   public Student(string name, int course) : base(name)
   {
      Course = course;
   }
}
```

Здесь в родительском классе Person есть конструктор, который позволяет присвоить объекту имя. В классе Student конструктор принимает 2 параметра, параметр name уже существует в конструкторе базового класса, поэтому с помощью base, просто ссылаемся и передаем параметр конструктору родительского класса. Поле course встречается только у класса Student, поэтому в конструкторе просто присваиваем параметр конструктора полю класса.

Ключевое слово base используется не только в конструкторах, но и в методах для обращения к реализации базового класса. Это особенно полезно, когда производный класс не заменяет, а расширяет поведение базового:

```
class Person
{
   public void SayName()
   {
      Console.WriteLine($"Имя: {Name}");
   }
}

class Student : Person
{
   public void PrintInfo()
   {
      SayName();  // Вызываем реализацию из Person
      Console.WriteLine($"Группа: {Group}");  // Добавляем своё
   }
}
```

Язык C# накладывает определённые ограничения на использование наследования, которые важно понимать.

1. C# поддерживает только одиночное наследование классов. Класс может иметь только одного непосредственного базового класса:

```
class A { }
class B { }
class C : A, B { }  // Ошибка: множественное наследование не разрешено
```

Это решение было принято сознательно для избежания конфликтов (например, если бы два базовых класса имели методы с одинаковыми именами), упрощения объектной модели и повышения читаемости кода. Однако стоит отметить, что множественное наследование интерфейсов в C# разрешено.

В данном коде представлена цепочка наследования:

```
class A
{
   public A()
   {
      Console.WriteLine("A");
   }
}

class B : A
{
   public B()
   {
      Console.WriteLine("B");
   }
}

class C : B
{
   public C()
   {
      Console.WriteLine("C");
   }
}
```

Технически в C# можно создавать длинные цепочки наследования. Однако на практике глубина иерархии больше 3-4 уровней часто становится тревожным сигналом. Чем глубже иерархия, тем сложнее становится сопровождение кода, отладка и понимание логики программы. Ошибки в глубоких иерархиях становятся менее очевидными, а тестирование — более сложным.

2. Иногда необходимо запретить дальнейшее наследование от класса. Для этого используется модификатор **sealed**:

```
sealed class FinalClass
{
   // Этот класс не может иметь наследников
}
class Derived : FinalClass { }  // Ошибка компиляции
```

Классы помечаются как sealed по разным причинам: логическая завершённость класса, соображения безопасности, архитектурные ограничения или оптимизация производительности.